Greedy Problem’s properties

- **Greedy Choice Property**: Choosing the best option at each phase can lead to a global (overall) optimal solution.
- **Optimal Substructure**: If an optimal solution to the complete problem contains the optimal solutions to the subproblems, the problem has an optimal substructure.

Problem list:

- Knapsack Problem
- Coin Change (minimum number of coins)
- Finding maximum value from a tree

Can we find a greedy solution for these problems?

Problem list:

- Knapsack Problem
- Coin Change (minimum number of coins)
- Finding maximum value from a tree

Can we find a greedy solution for these problems?

NO

Coin Change

Number of coins 3: {1,3,4}, Target value: 6

Greedy Choice (choose the highest valued coins):
4 + 1 + 1

Optimal Solution: 3+3

0/1 Knapsack

Number of objects 3:

Weight: {1,2,3}
Value: {6,10,12}

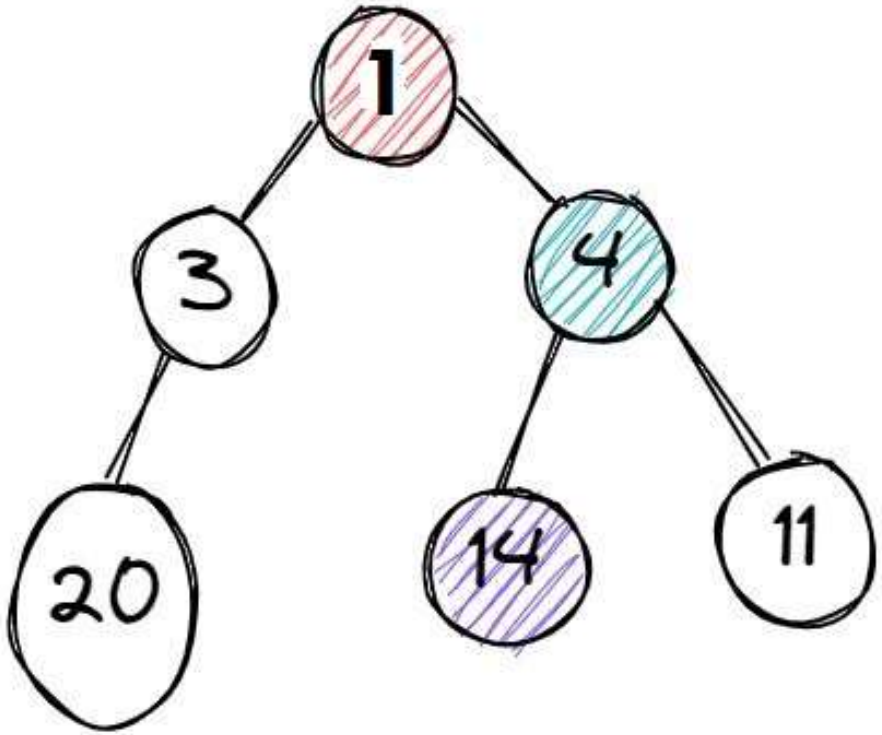
Value/Weight: {6, 5, 4}

Total Capacity: 5

Greedy Choice (choose the object with best value/weight ratio):
object1 + object2 (6+10): **16**

Optimal Solution: object2 + object 3 (10 + 12): **22**

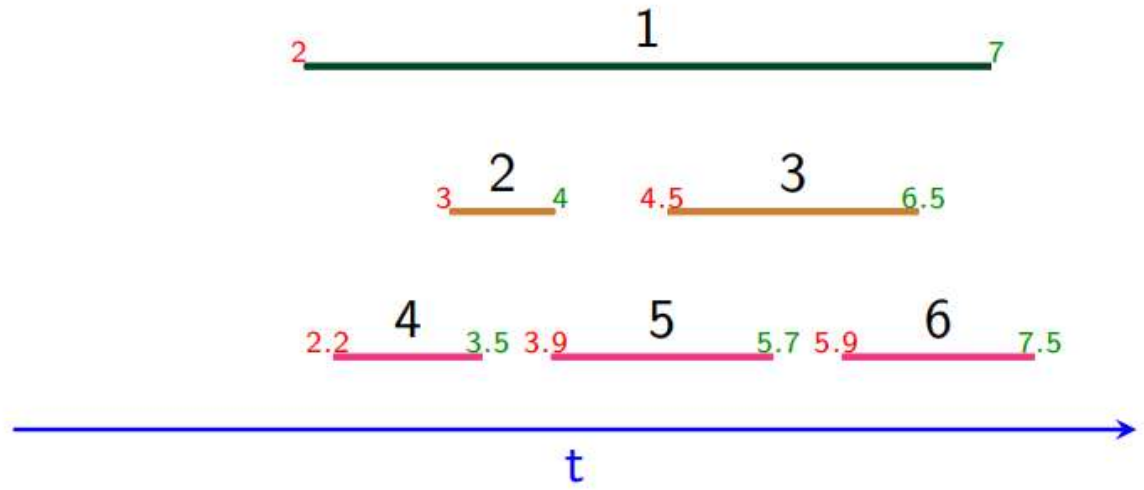
Maximum Value from a Tree



Problems for Greedy Algorithm

- Fractional knapsack problem
- Interval scheduling
- Job Scheduling
- Interval partitioning
- Huffman Encoding

Interval Scheduling (Activity selection)



- Single resource
- n requests
- Each with $s[i]$ start time and $f(i)$ finish time ($s[i] < f[i]$)
- Two requests i and j are compatible if they don't overlap
  - $f(i) \leq s(j)$, or
  - $f(j) \leq s(i)$
- **Goal:** Select a compatible subset of requests of maximum size

Greedy solution idea

- Sort the requests
- Select the requests from the sorted list, that does not overlap with the already selected requests

Interval Scheduling (Sorting Options)

- Earlier start time
- Smallest request
- Minimum overlap (fewest conflicts)
- Earlier finish time

Interval Scheduling (Sorting Options)

- Earlier start time
  - (1,6) (2,3) (4,5)
- Smallest request
- Minimum overlap (fewest conflicts)
- Earlier finish time

Interval Scheduling (Sorting Options)

- Earlier start time
  - (1,6) (2,3) (4,5)
- Smallest request
  - (5,8) (1,6) (7,15)
- Minimum overlap (fewest conflicts)
- Earlier finish time

Interval Scheduling (Sorting Options)

- Earlier start time
  - (1,6) (2,3) (4,5)
- Smallest request
  - (5,8) (1,6) (7,15)
- Minimum overlap (fewest conflicts)
  - (1,3) (4,6) (7,9) (10,12) (5,8) (2,5) (2,5) (2,5) (8,11) (8,11) (8,11)

Counter examples

counterexample for earliest start time

counterexample for shortest interval

counterexample for fewest conflicts

Interval Scheduling (Activity Selection)

Sort all activities by their end times (ascending).

Initialize last_end_time = 0

For each activity (start, end) in sorted list:

    If start >= last_end_time:

        Select the activity

        Update last_end_time = end

Proof of correctness

Let G, be the set of selected intervals by greedy algorithm

and

O, be the set of optimal intervals

If G = O, then G is already optimal. Nothing to prove.

Proof of correctness

Assume, O != G

Let,

G = $\{g_1, g_2, \dots, g_m\}$

O = $\{r_1, r_2, \dots, r_n\}$

Here, n >= m

Proof of correctness

Assume k^{th} element is the first mismatched interval between O and G

G = {$g_1, g_2, g_{k-1}, g_k, \ldots, g_m$}

O = {$g_1, g_2, g_{k-1}, r_k, \ldots, r_n$}

Can we replace r_k with g_k without contradiction and O still remain Optimal?

Proof of correctness

As both of g_k and r_k is compatible with g_{k-1}, so different start time does not create any conflict

As, $f_{gk} <= f_{rk}$

So we can replace r_k with g_k without creating conflict with r_{k+1}

After replacement:

G = {$g_1, g_2, g_{k-1}, g_k, \ldots, g_m$}

O = {$g_1, g_2, g_{k-1}, g_k, \ldots, r_n$}

Proof of correctness

By repeating this process, we can replace all g_i to the set O

G = {$g_1, g_2, g_{k-1}, g_k, \dots, g_m$}

O = {$g_1, g_2, g_{k-1}, g_k, \dots, g_m, \dots, r_n$}

But this can't be true, as Greedy algorithm keeps adding intervals, if there is no conflict. So if there are non-conflicting elements is O after g_m, that should be picked by greedy algorithm and would be part of G.

So |G|=|O|, and we can convert G to O, thus proving the optimality of G

Job sequencing

- Given an **array of jobs** where every job has:
  - A deadline
  - Associated profit if the job is finished before the deadline.
- It is also given that every job takes a single unit of time, so the minimum possible deadline for any job is 1.

Target is to **maximize the total profit** if only one job can be scheduled at a time.

Input: Four Jobs with following deadlines and profits

JobID	Deadline	Profit
a	4	20
b	1	10
c	1	40
d	1	30

Output: Following is maximum profit sequence of jobs

c, a

Input: Five Jobs with following deadlines and profits

JobID	Deadline	Profit
a	2	100
b	1	19
c	2	27
d	1	25
e	3	15

Output: Following is maximum profit sequence of jobs

c, a, e

Greedy Solution of Job sequencing

Idea is to select the most profitable job, but complete it as delayed as possible

- Sort all jobs in decreasing order of profit.
- Iterate on jobs in decreasing order of profit. For each job , do the following :
  - Find a time slot i, such that slot is empty and i < deadline and i is greatest.Put the job in this slot and mark this slot filled.
  - If no such i exists, then ignore the job.

Proof of correctness

Let profit and deadline of a job i, (P_i, d_i). 1 <= i <= n

Let I be the set of jobs selected by the greedy method.

Let J be the set of jobs in an optimal solution.

We need to show that both I and J have the same profit values and so I is also optimal.

Proof of correctness

We can assume I != J as otherwise we have nothing to prove.

Note that if $J \subset I$, then J cannot be optimal.

Also, the case $I \subset J$ is ruled out by the greedy method.

So, there exist jobs a and b such that $a \in I$, $a \notin J$, $b \in J$, and $b \notin I$.

Let a be the highest-profit job such that $a \in I$ and $a \notin J$.

If $P_b > P_a$, then the greedy method would consider job b before job a and include it into I.

So: $P_a >= P_b$ for any b in J

Proof of correctness

Let S_i and S_j are feasible schedule for I and J

Let i be a job such that $i \in I$ and $i \in J$. Let i be scheduled from t to t + 1 in S_i and t' to t'+ 1 in S_j.

If t < t’,

We can interchange job i, with the job (if any) scheduled in [t', t' + 1] in S_i. If no job is scheduled in [t', t' + 1] in I, then i is moved to [t', t' + l].

If t' < t:

then a similar transformation can be made in S_j.

Proof of correctness

For example: Job i is scheduled in slot 2 in I, and slot 4 in J ($t < t'$)

So, i can be moved to slot 4 in S_i, as it is already in slot 4 in S_j.

If there is a job in slot 4 in S_i, we can move it to slot 2 (as 2 is even better option than 4).

If there is no job in slot 4 in S_i, then we can move job i to slot 4.

Proof of correctness

In this way, we can obtain schedules S_i' and S_j' with the property that all jobs common to I and J are scheduled at the same time.

Proof of correctness

Consider the interval $[t_a, t_a + 1]$ in S_i' in which the job a (defined earlier) is scheduled.

Let b be the job (if any) scheduled in S_j' in this interval.

As, $P_a >= P_b$, so replacing b with a, will not reduce the total profit.

So: J' = J - {b} U {a}. J' has the profit value no less than of J

By repeatedly using the transformation just described, J can be transformed into I with no decrease in profit value. So I must be optimal.

Interval Partitioning

Input A set of lectures, with start and end times

Goal Find the minimum number of classrooms, needed to schedule all the lectures such two lectures do not occur at the same time in the same room.

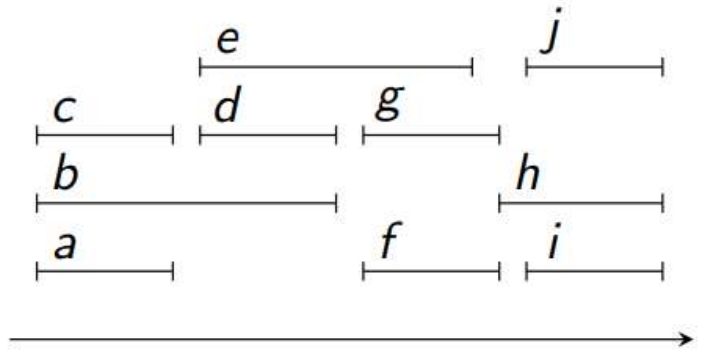


Figure: A schedule requiring 4 classrooms

Solution Idea

- Apply interval scheduling, and assign a room for the selected lectures
- Apply interval scheduling for the remaining lectures and add a new room. (do this until there are no lectures remaining)

- Will it work?

Solution Idea

- Apply interval scheduling, and assign a room for the selected lectures
- Apply interval scheduling for the remaining lectures and add a new room. (do this until there are no lectures remaining)

- Will it work? (NO)

Counter example

(3,7) (5,9) (8,11) (12,13) (10,14) (16,18) (13,20)

- (3,7) (8,11) (12,13) (16,18)
- (5,9) (10,14)
- (13,20)

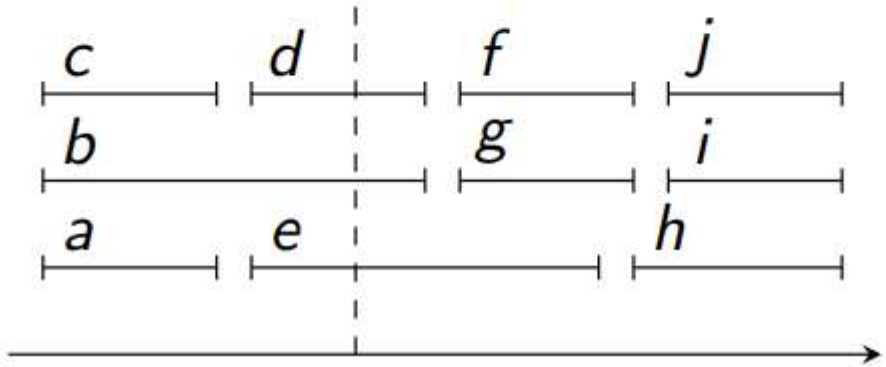
Greedy Solution

Sort all lectures by their starting time

```
set: r = 0
for j=1 to n
        if(there is a room k, which is compatible with j)
                assign j to room k
        else
                allocate a new room r = r+1
                assign j to r+1
```

Proof of correctness

- For a set of lectures R, k are said to be in conflict if there is some time t such that there are k lectures going on at time t.
- The depth of a set of lectures R is the maximum number of lectures in conflict at any time.



Proof of correctness

For any set R of lectures, the number of class-rooms required is at least the depth of R.

Proof.

All lectures that are in conflict must be scheduled in different rooms. □

Proof of correctness

Let d be the depth of the set of lectures R. The number of class-rooms used by the greedy algorithm is d.

Proof.

- Suppose the greedy algorithm uses more that d rooms. Let j be the first lecture that is scheduled in room $d + 1$.
- Since we process lectures according to start times, there are d lectures that start (at or) before j and which are in conflict with j.
- Thus, *at the start time of* j, there are at least $d + 1$ lectures in conflict, which contradicts the fact that the depth is d. □